Scientific QA Evidence

An independent QA audit of OpenAI's Lean proofs for Navier–Stokes alternatives (C) and (D) and the Euler singularity, with Lean bridges to Clay's text and checker controls · Report for release **v1.3.0** · 21 October 2026

Author: **Cristian Guerra**, QA Automation, Medellín, Colombia · github.com/criguex/scientific-qa-evidence

Read this first. The proofs belong to OpenAI (`openai/NavierStokesAndEuler`). This work proves nothing new, claims no Millennium Prize result and is not a mathematical review. It is a software-QA study: rebuild the published artifacts, recheck them with independent checkers, test whether the checking pipeline itself catches errors, and publish every step with hashes.

1. Conclusions

1. **OpenAI's (C), (D) and Euler proofs check, repeatedly and from scratch.** Comparator, with nanoda and the Lean kernel, accepted both challenges on five different builds, the last two compiled entirely from source with nothing prebuilt (`lake --no-cache build`, exit 0, 11,424 jobs, 1 h 29 min and 1 h 16 min). A fresh `leanchecker` replay agreed, and the kernel source used includes every known 2026 kernel soundness fix.
2. **Comparator checks the exact statement.** It refused all six one-symbol changes to the challenge and accepted a comment-only control. The challenge's 18 declarations match Google DeepMind's public Formal Conjectures statement after textual normalization (name resolution not verified).
3. **OpenAI's theorems imply Clay's (C) and (D) as written literally in Lean,** for every $\nu > 0$, with no hypotheses and only the standard axioms, under both readings of smoothness at $t = 0$ (`k1_C`, `k1_D`, `k1_C_oneSided`, `k1_D_oneSided`). This is relative to the literal rendering `ClayLiteral`, which nobody independent has reviewed against the PDF; (D) reads the erratum on periodic pressure as a solution condition, and (C) does not use it.
4. **A refusal is not a refutation.** On three Mathlib theorems, Comparator refused all eight changed statements, including three equivalent ones and a definitionally equal one. A second route in Lean, with a guard against using the theorem under test, classified each as equivalent, weaker or false, all as pre-registered.
5. **Not settled here.** Whether the Lean proof follows OpenAI's paper is out of scope and is disputed by Bastounis, Circelli and Hansen (arXiv:2610.08144). No mathematician has reviewed the bridges.
6. **What the protocol adds.** In this case it surfaced eight findings that the documented verification procedure does not produce, three of them corrected defects in this study's own work. In a pre-registered benchmark its checks flagged four planted fault classes that the baseline passes. Four review rounds found and fixed 17 issues in that experiment itself.
7. **Not claimed.** Methodological novelty (a targeted literature search found close prior work), a real-world detection rate, or any part of the mathematical proof.

2. Formal results

Question	Result	Limitation
Does it build from source?	B1 (2026-10-20) exit 0; B3 exit 0 (Mathlib and project local, 7 small dependencies prebuilt from Reservoir); **B4** `--no-cache`	Lean toolchain and checker binaries reused; one Linux host

	exit 0, every `.olean` newer than the build start; **B5 `--no-cache` (for v1.3.0) exit 0, 11,424 jobs, 1 h 16 min, nothing prebuilt**	
Do independent kernels accept it?	Comparator with nanoda and the Lean kernel accepted Navier–Stokes (C, D) and Euler in all five runs, one per build	Standalone Comparator `19e111e`; one host
Fresh replay	`leanchecker --fresh` exit 0 for both solutions (19.6 and 22.5 min)	Same Lean kernel; lean4lean could not be built for Lean 4.34 and never examined the proofs
Statement mutants	6/6 refused (*statements do not match*); comment-only control accepted	Tests statement equality only; says nothing about (A)/(B)
Kernel currency	Lean `src/kernel` tree of v4.34.0-rc2 identical to v4.33.1–v4.34.1 (all 2026 kernel fixes); nanoda postdates its fix; a public exploit for #14576 is refused by the kernel used	Known bugs only; v4.34.1 runtime fixes not included
Statement provenance	18 declarations match DeepMind `8bf45ed` after textual normalization	Textual, not semantic
Clay risk review	16 clauses: 6 direct, 9 via standard facts, 1 needing a human decision	Reading-based; not reviewed by a mathematician

3. Numerical mutation campaign

Nine synthetic models × three configurations, independent oracles, frozen contract (tolerance 10^{-6}). Per host: **27 PASS, 54 KILLED, 3 SURVIVED**, identical on macOS arm64 and Linux x86_64 (maximum scaled difference 1.11×10^{-16}). The 54 KILLED are 51 distinct behaviors; the 3 survivors are a deliberately tiny RC change and stay visible. A rerun reproduces all 84 saved records. Synthetic models, not physical experiments.

4. What the protocol found that the documented procedure does not show

Baseline: OpenAI's README procedure (`lake exe cache get`, `lake build`, Comparator).

#	Finding	Protocol element
1	A build believed clean had silently received 7 prebuilt dependencies; the log did not show it	Artifact timestamps vs build start
2	A full from-source build is possible and verified	`--no-cache` + artifact check
3	Two runs reported as "same build" used different builds	Per-run receipts with original hashes
4	Comparator refuses any one-symbol statement change	Statement mutants with a negative control
5	Challenge text matches the public statement	Statement provenance
6	Kernel and nanoda current with 2026 fixes	Kernel currency check
7	Periodic pressure in (D) rests on reading Clay's erratum	Directional risk review
8	The numerical runner reported a valid oracle when the oracle had failed	Fault injection into the test harness

Findings 1, 3 and 8 were defects in this study's own work, caught by the protocol and corrected in the open. One case is not a rate.

5. Pre-registered blind-fault benchmark

Seed committed by hash before the design was frozen; design and scripts frozen by hash; 21 cases drawn by the author; blind results committed before the key; key and seed revealed, and the draw reproduced byte for byte. After review it is presented as a **deterministic regression benchmark**; commit order is not taken as proof of blinding.

Planted fault	Baseline	Protocol
Clean	**passes**	**passes**
F1 · prebuilt artifacts, clean log	passes	**flags**
F2 · statement differs textually from the reference	passes	**flags**
F4 · oracle failure hidden by the harness	passes	**flags**
F5 · receipt names a log the build did not write	passes	**flags**
F6 · positive control (`sorry`)	**flags**	**flags**

Review round	Found	Outcome
1 · external	7	Errata (framing, F2 is a textual difference not a weakening, cost, seed) and evaluator v2 (fail closed on absent/incomplete evidence). The v1 "0 false alarms" does not hold.
2 · external	3	v2.1: non-object JSON, header-only tables, non-zero exit codes
3 · fuzzing	0	8,000 evaluations, no crash; same fuzzer finds 52 crashes in v2

4 · semantic attacks	7	v2.2: 5 false accepts and 2 false alarms fixed

Limits: planted faults over recorded artifacts, not live Lean; not a real-world rate; each fault sits in an artifact the baseline does not read. Class F3 (kernel exploit) was withdrawn before freezing after an automated safety filter stopped the assistant's work on it; the reason is recorded.

6. From OpenAI's theorem to Clay's text

To get Clay's (C) and (D) from OpenAI's formal theorem, one direction per clause is enough: OpenAI's data must meet Clay's conditions, and any Clay solution must be a formal solution. Clay's equations (1)–(11), (C), (D) and the erratum are written literally in Lean, in coordinates and with one-sided time derivatives (`ClayLiteral.lean`); `Bridges.lean` proves each clause and composes them.

Clay's text	OpenAI's formal version	Lean
u° smooth, div u° = 0, decay (4)	OpenAI's witness has $u_0 = 0$	`d1_zero`, `d2_zero`, `d3_zero`
f smooth on $\mathbb{R}^3 \times [0, \infty)$, either reading at t = 0	OpenAI's force is smooth on all of $\mathbb{R} \times \mathbb{R}^3$, its own extension	`witness_C`, `witness_D`
decay (5), (9) of each mixed partial	bound on the norm of the m-th derivative	`d5`, `d7` via `clay_bound`
(1) in coordinates; (2); (3)	vector equation with Mathlib's Δ and gradient; trace of the Jacobian; initial condition	`s1`, `s2`, `s3`
(6), (11) smoothness; (7) Lebesgue energy bound	`ContDiffOn` on the closed half-space; `MemLp 2` and Bochner bound	`s4`, `s5`
(8), (10), erratum: periodicity	`IsOnePeriodic`	`d6`, `s6`, `s7_formal_side`

State: 16 of 17 claims proved in Lean; K0 written and checked non-vacuous (the zero solution meets every Clay solution condition); S7 interpretive. All theorems use only `propext`, `Classical.choice` and `Quot.sound`. **Open:** an independent review of `ClayLiteral` against the PDF (14 points, smoothness at t = 0 first; `SmoothOneSided` is a definition and its name proves nothing), Clay's answer on periodic pressure (D only), and a mathematician's review.

7. Mathlib controls: a changed statement is not a false one

Pre-registered (commit `046a128`, amended once before any run after outside review). Correct statements, reformulations and deliberate errors for three theorems Mathlib already proves.

Theorem	Statements tried	Comparator	Second route in Lean
Infinitude of primes	correct; `n + 1 ≤ p`; `n ≤ p`; only n ≤ 100; quantifiers swapped	accepted; refused ×4	equivalent (`Iff.rfl`); equivalent (for every predicate); weaker (both directions); false
Cauchy–Schwarz	correct; squared form; strict <	accepted; refused ×2	equivalent; false (x = y = 1)
Dominated convergence	correct; `Integrable` unfolded; domination removed	accepted; refused ×2	equivalent only because domination gives bound ≥ 0; false ((n+1) on (0, 1/(n+1)))

All 11 cases and both controls (a cheating equivalence rejected by the dependency guard; a signed bound showing the DCT reformulation needs domination) came out as registered. Every step has a receipt with command, input and output hashes and exit code; `verify.py` checks the records against them and has

regression tests for two verifier defects found by an outside reviewer. Limits: author-designed cases, no held-out cases yet, three theorems only.

8. Reproducibility and traceability

- Release **v1.3.0**, content commit `b3fd34d`; CI green on Ubuntu and macOS, Python 3.9 and 3.12.
- **Validation from a fresh clone:** every hash, rerun, consistency check and test passes on macOS; on the Linux host the Lean bridges compile from the clone into an empty directory and all 21 Mathlib-control steps reproduce their exit codes and input hashes (`evidence/FROM_ZERO.md`). A fifth build, B5, repeated the full build from source and Comparator accepted both challenges again (`evidence/formal/b5/`).
- Every cited repository, pinned commit, path, paper and URL was checked against its source; `CITATION.cff` lists them with revisions and licenses and validates against CFF 1.2.0.
- Every curated file has a SHA-256 in `evidence/SHA256SUMS.txt`; every public log is traced to its original's hash in `PROVENANCE.md`.
- `REPRODUCE.md` separates what is recomputed (numerical campaign, statement comparison, benchmark) from what is only verified by hash (formal logs).
- Related work, findings, claims with limitations, and errata are all in the repository: `RELATED_WORK.md`, `FINDINGS.md`, `CLAIMS.md`, `experiments/blind-faults/`.
- AI assistance is disclosed in `AUTHORSHIP.md`, including a mid-session change of the assistant's reported model; human authorship and responsibility remain with the author.

Contact: Cristian Guerra · github.com/criguex · Repository: https://github.com/criguex/scientific-qa-evidence